

QUARK ENGINE

Spécification & Architecture de l'Agent Codeur Zero-Token

Dossier Synthétique d'Architecture Systèmes & Spécifications IA

Vision Globale : Quark Engine est une norme d'architecture et un agent de développement hybride conçu pour réduire de 99.9 % la consommation de tokens d'entrée. Il bascule le paradigme du codage traditionnel vers un Spec-Driven Development (SDD) articulé autour du langage naturel structuré et de la transpilation AST locale.

1. Le Problème du Modèle Classique

Les agents de codage actuels (Aider, Claude Code, Cursor) souffrent d'un goulot d'étranglement économique et technique sévère :

- Explosion du Contexte** : À chaque modification, l'intégralité du code source (jusqu'à 100 000 tokens) est ré-injectée dans la fenêtre de contexte de l'API.
- Bruit Syntaxique Verbeux** : En Flutter/Dart, une grande partie du fichier est composée de structures répétitives (BuildContext, setState, fermetures de parenthèses) qui consomment des tokens à forte valeur sans apporter de logique métier.
- Coûts Incompatibles avec l'Illimité** : Les modèles distants facturent chaque token d'entrée, rendant les abonnements à prix fixe non viables sur les gros modèles propriétaires.

2. L'Architecture en 4 Briques de Quark Engine

Quark Engine résout ce problème en séparant strictement la réflexion logique (en texte léger) de la génération de syntaxe brute (exécutée en local).

Brique	Nom de Composant	Rôle dans l'Architecture
Le DSL / Format	Quark Spec (.qrk)	Langage naturel structuré ultra-dense (JSON/YAML/DSL).
Le Parser / Compresseur	Quark Core	Extraction AST local (Dart analyzer) + SLM Local.
Le Séquenceur	Quark Router	Routeur sémantique local interceptant l'intention utilisateur.
L'Invocateur	Quark Pulse	Appel en Single Pass ultra-court vers l'agent distant (Claude).

3. Le Flux de Travail (Zero-Token Pipeline)

- Intention en Langage Naturel (« Le Roman »)** : L'utilisateur décrit l'application ou la modification sous forme d'intentions et de règles métier précises.
- Compression AST & Génération de Spec (Quark Core)** : Un parser local analyse le code Dart existant via l'AST et le résume en une spécification Quark Spec ultra-compacte. Le bruit syntaxique est éliminé.
- Aiguillage Sémantique (Quark Router)** : Un modèle local très léger (SLM) ou un routeur d'embeddings vérifie la complexité. Les demandes simples sont traitées localement.

4. **Invocateur en Passe Unique (Quark Pulse)** : Seule la Quark Spec de 100 à 200 tokens est envoyée à l'agent distant. Claude génère la modification en une seule passe sans lire le projet complet.
5. **Compilation et Validation Locale** : Un script local convertit la réponse en code `.dart` valide et exécute la suite de tests unitaires sur le disque.

4. Analyse Comparative & Gain de Performance

Métrique	Agent Traditionnel	Quark Engine
Tokens d'entrée / requête	50 000 - 100 000 tokens	100 - 500 tokens
Erreurs de syntaxe Dart	Fréquentes (arbres UI profonds)	Quasi-nulles (Transpilation AST)
Dépendance à l'API distante	Élevée (Allers-retours continus)	Minimale (Single Pass ciblé)
Réduction de coût globale	Baseline (100 %)	-99.9 % de réduction

5. Feuille de Route pour l'Établissement du Standard

- Phase 1 — Prototype (POC) : Développement du parser AST Dart (analyser) et du convertisseur bidirectionnel Dart ↔ Quark Spec.
- Phase 2 — Benchmarks : Publication des résultats comparatifs de consommation de tokens sur des cas d'usage Flutter réels.
- Phase 3 — Open Source & Distribution CLI : Déploiement du package CLI `quark` sur GitHub et promotion auprès de la communauté des développeurs IA.